

Modelos de Arquitectura

Los modelos de arquitectura nos permiten definir un patrón de diseño antes de iniciar la implementación de un componente, ya sea de hardware o de software. A partir de este momento, nos centraremos en el contexto de co-diseño de hardware/software usando descripción de alto nivel en SystemC.

Un modelo de arquitectura describe:

- Componentes del sistema
- Comunicación interna de los componentes
- Datapath
- Funcionalidad de cada componente
- Restricciones: desempeño, energía o paralelismo

En ESL, el objetivo es trabajar en un nivel de abstracción alto para explorar diseños rápidamente (usando exploración del espacio de diseño), validar el comportamiento y decidir las particiones hardware/software, permitiendo el desarrollo en paralelo.

Niveles de Abstracción

Podemos modelar una arquitectura a partir de distintos enfoques o niveles de abstracción. Es importante recordar un par de conceptos:

- Arquitectura: se refiere a la parte de qué hace el sistema. Ejemplo: x86 ISA
- Microarquitectura: se refiere a cómo se implementa el sistema para cumplir con el qué hace. Ejemplo: AMD Zen 3

a) Modelo Funcional

Describe qué hace (arquitectura) el sistema a nivel de operaciones sin por implementación (microarquitectura).

Destaca las principales características de funcionalidad, lo que toma en cuenta:

- Algoritmo
- No modela tiempo, solo la función que ejecuta
- Es ideal para validación temprana, para validar si el procesamiento es adecuado

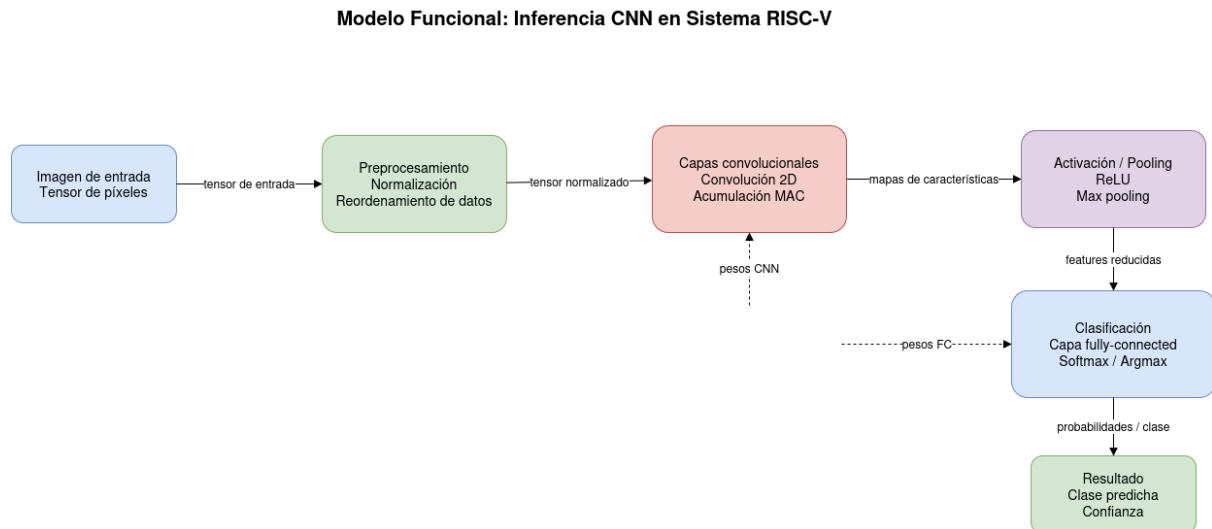


Figura 1. Modelo funcional. Elaborado con ayuda de la IA

Puede verse como una descripción de software de un posible módulo de hardware, como será el del proyecto (CGRA, GPU, CPU).

En este caso, puede ser un modelo de C/C++ o Python.

b) Modelo Arquitectónico

Toma en cuenta el modelo funcional, pero adicionalmente introduce aspectos de implementación desde una perspectiva todavía abstracta, como la presencia de bloques como procesadores, memorias, buses, aceleradores y paralelismo.

El modelo arquitectónico describe cómo se organiza el sistema. Un ejemplo es un sistema con:

- CPU RISC-V
- DMA
- Memoria DDR
- Acelerador CNN

Modelo Arquitectónico: Sistema RISC-V + CNN Accelerator

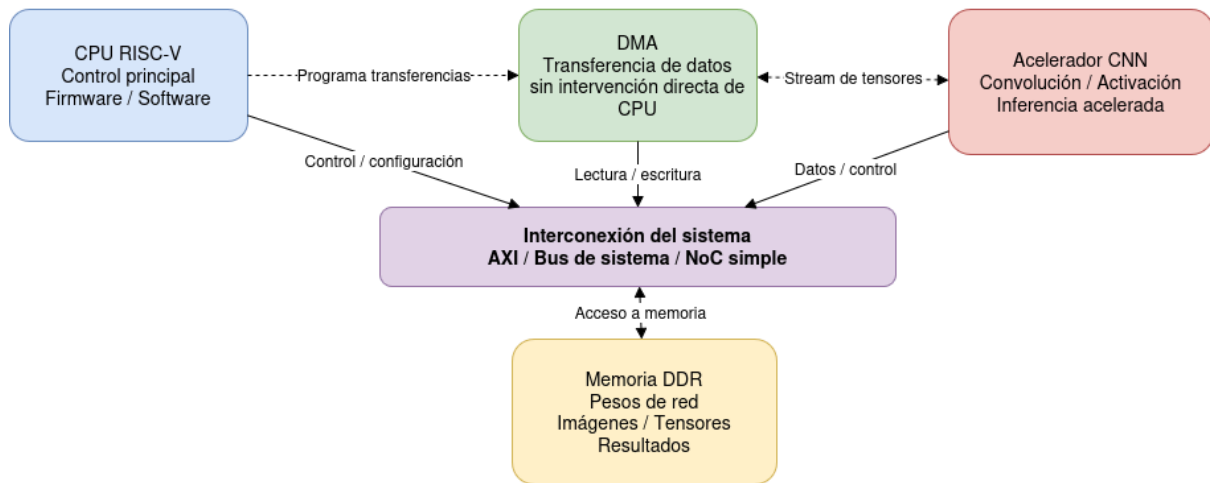


Figura 2. Modelo Arquitectónico. Elaborado con ayuda de la IA

En este caso, podemos todavía implementarlo a alto nivel con C/C++/Python, solo que ahora cada módulo va a ser una tarea y su funcionamiento es casi independiente. Esto es relevante para la partición HW/SW (establecer un modelo que pueda ser usado para desarrollo de SW y continuar con el HW), topología, interconexión.

En este caso, será el modelo de SystemC.

c) Modelo Ciclo-Preciso

Baja el nivel de abstracción del modelo arquitectónico, agregando el soporte de tiempo en sustitución del modelo de eventos por transacción.

Incluye relojes, latencias, pipelines, stalls y throughput. Esto nos permite modelar mejor el rendimiento de la arquitectura sin aún tomar en cuenta *timing* por los límites físicos. Se puede considerar como un modelo de presíntesis, donde nos concentramos en la implementación pero sin la entrada o el feedback de la parte física, que incluye retrasos.

En este caso, será el modelo extendido de SystemC.

d) RTL (Register Transfer Level)

Esto acerca el modelo al hardware real. Ya se tiene que traducir el modelo de SystemC a un modelo de HDL, donde tenemos que pensar más en una implementación de hardware. Describe: registros, FSMs, señales, lógica combinacional y secuencial.

Los lenguajes que se utilizan son típicamente Verilog, VHDL y SystemVerilog. Ya a partir de esta descripción, se puede sintetizar el hardware, permitiendo trasladar el diseño en implementaciones dedicadas a un ASIC o FPGAs, lo que nos permite modelar aspectos más físicos como:

- Timing closure
- Frecuencias máximas
- Consumo de recursos
- Consumo de potencia

Tarea: Leer los capítulos 2, 4 y 6 del libro.

Tipos de modelos de arquitectura

Los modelos de arquitectura nos permiten partir de un punto donde se define cómo se organizan los elementos computacionales, la comunicación y la memoria.

a) Arquitectura Von Neumann

La arquitectura Von Neumann es un modelo de arquitectura de computación tradicional donde la memoria es compartida y almacena las instrucciones y los datos.

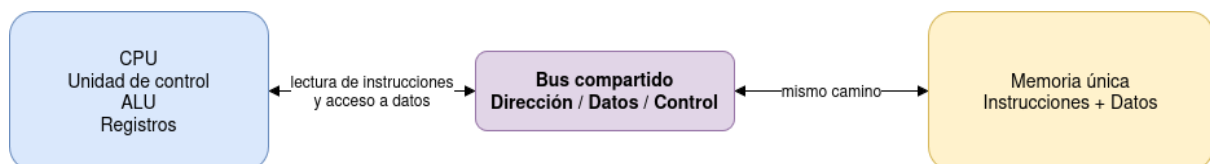


Figura 3. Modelo de Von Neumann. Elaborado con ayuda de la IA.

Son simples pero tienen un serio cuello de botella con respecto al paralelismo de leer/escribir datos con instrucciones.

b) Arquitectura Harvard

Separa la memoria de datos y de instrucciones, lo que resuelve el cuello de botella de Von Neumann. Sin embargo, su complejidad incrementa al manejar dos espacios de direcciones y dos memorias distintas. Sus usos principales incluyen DSPs.

Arquitectura Harvard

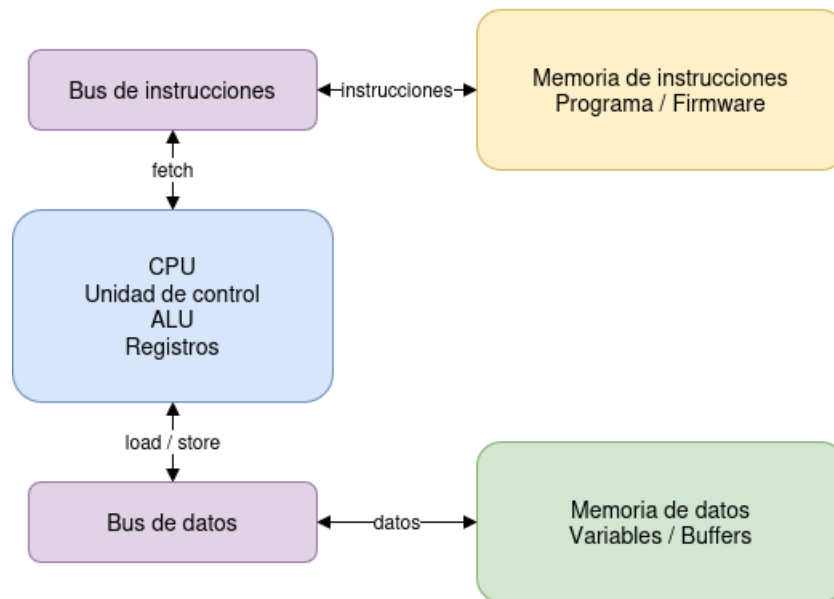


Figura 4. Modelo de Harvard. Elaborado con ayuda de la IA.

c) Arquitecturas Paralelas

En este caso, los modelos de Harvard y Von Neumann poseen una única unidad de procesamiento, lo que limita el rendimiento. Las arquitecturas paralelas utilizan múltiples unidades de procesamiento. Algunos ejemplos de arquitecturas paralelas son los multi núcleos, GPU y arreglos sistólicos.

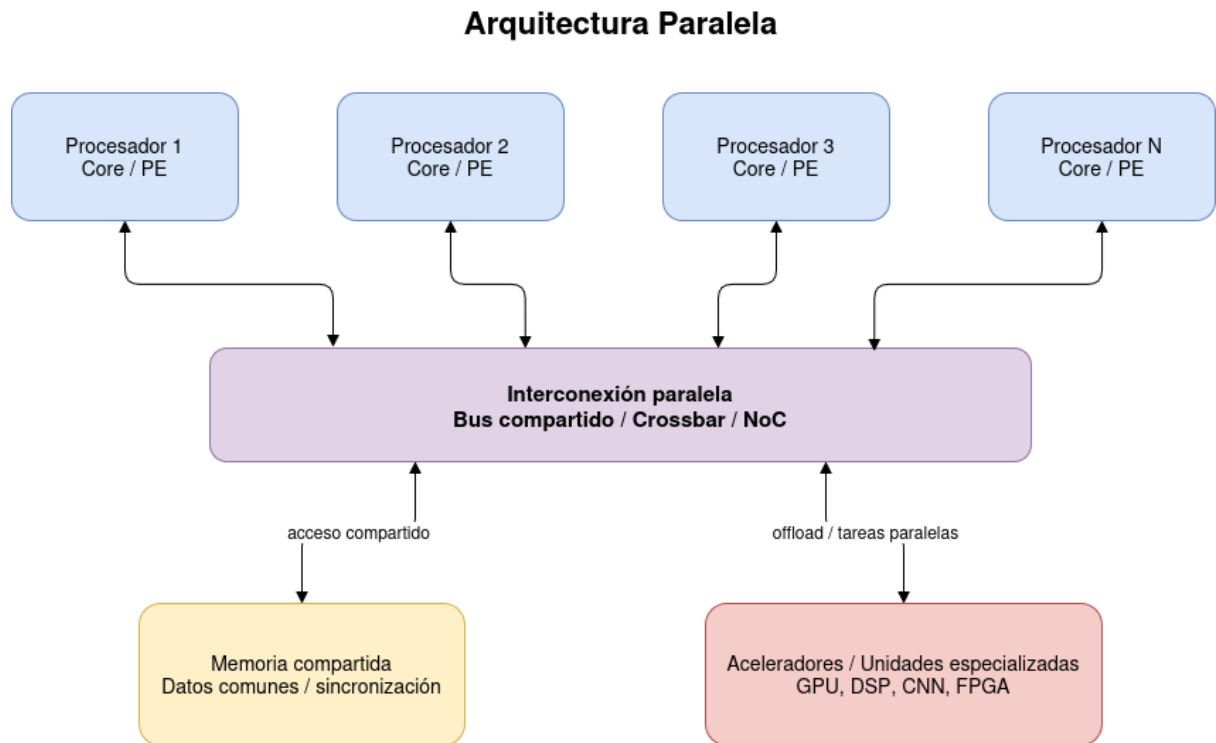


Figura 5. Modelo de arquitectura paralela. Elaborado con ayuda de la IA.

Modelado de comunicación en ESL

En un modelo arquitectónico no basta con decir que existe una CPU, una memoria o un acelerador. También se debe describir cómo se comunican, qué tipo de datos intercambian, con qué latencia, con qué ancho de banda y bajo qué protocolo.

El modelado de la comunicación también sigue el patrón del modelo de arquitecturas, desde la parte arquitectónica, ciclo preciso hasta RTL.

a) Comunicación por transacciones

Este es el enfoque más típico de ESL/TLM. Es utilizado para el modelado arquitectónico (usando SystemC) y de ciclo preciso. Tomando en cuenta las señales de una interfaz X, en vez de modelar cada señal:

```
addr = 0x1000
```

```
write = 1
valid = 1
ready = 1
```

El modelo arquitectónico y de ciclo-preciso en SystemC, se modela una operación abstracta:

```
write(0x1000, data)
read(0x2000)
send_tensor(buffer)
```

Esto permite simular más rápido y enfocarse en el comportamiento del sistema, en lugar de detalles de implementación.

b) Comunicación por buses o hubs

Aquí los bloques se comunican mediante una interfaz común en un bus. Sin embargo, puede haber enlaces de comunicación que son en bus, con distintas topologías. En este caso, el modelo arquitectónico se enfoca en cómo se comunican los componentes y por dónde viajan las transacciones. En el modelado de ciclo-preciso, se debe pensar en cómo y cuándo circulan los datos dentro de las interfaces y estructuras de red.

- **Comunicación punto a punto:** consiste en enlaces dedicados entre dos dispositivos. Ofrece baja latencia y alta velocidad, ya que no comparte el medio de comunicación con otros módulos. Sin embargo, escala mal cuando aumenta la cantidad de componentes debido al incremento del cableado y conexiones.
- **Comunicación basada en bus:** todos los dispositivos comparten un canal común de comunicación. Es una solución simple y ampliamente utilizada en sistemas embebidos y SoCs, aunque puede generar cuellos de botella cuando varios módulos intentan acceder al bus simultáneamente.
- **Comunicación en malla (mesh):** los nodos se conectan con sus vecinos formando una grilla. Esta topología permite múltiples rutas de comunicación y mejora la escalabilidad, siendo común en arquitecturas paralelas y sistemas manycore.
- **Comunicación en anillo:** los dispositivos se organizan en un circuito cerrado donde los datos circulan entre vecinos. Reduce la complejidad de conexión respecto a una

mallla completa, aunque la latencia aumenta conforme los datos deben atravesar más nodos.

- **Network-on-Chip (NoC):** modela la comunicación interna del chip como una red de paquetes con routers y enlaces. Es una solución altamente escalable y eficiente para sistemas complejos con múltiples núcleos y aceleradores, ampliamente utilizada en SoCs modernos y arquitecturas de inteligencia artificial.

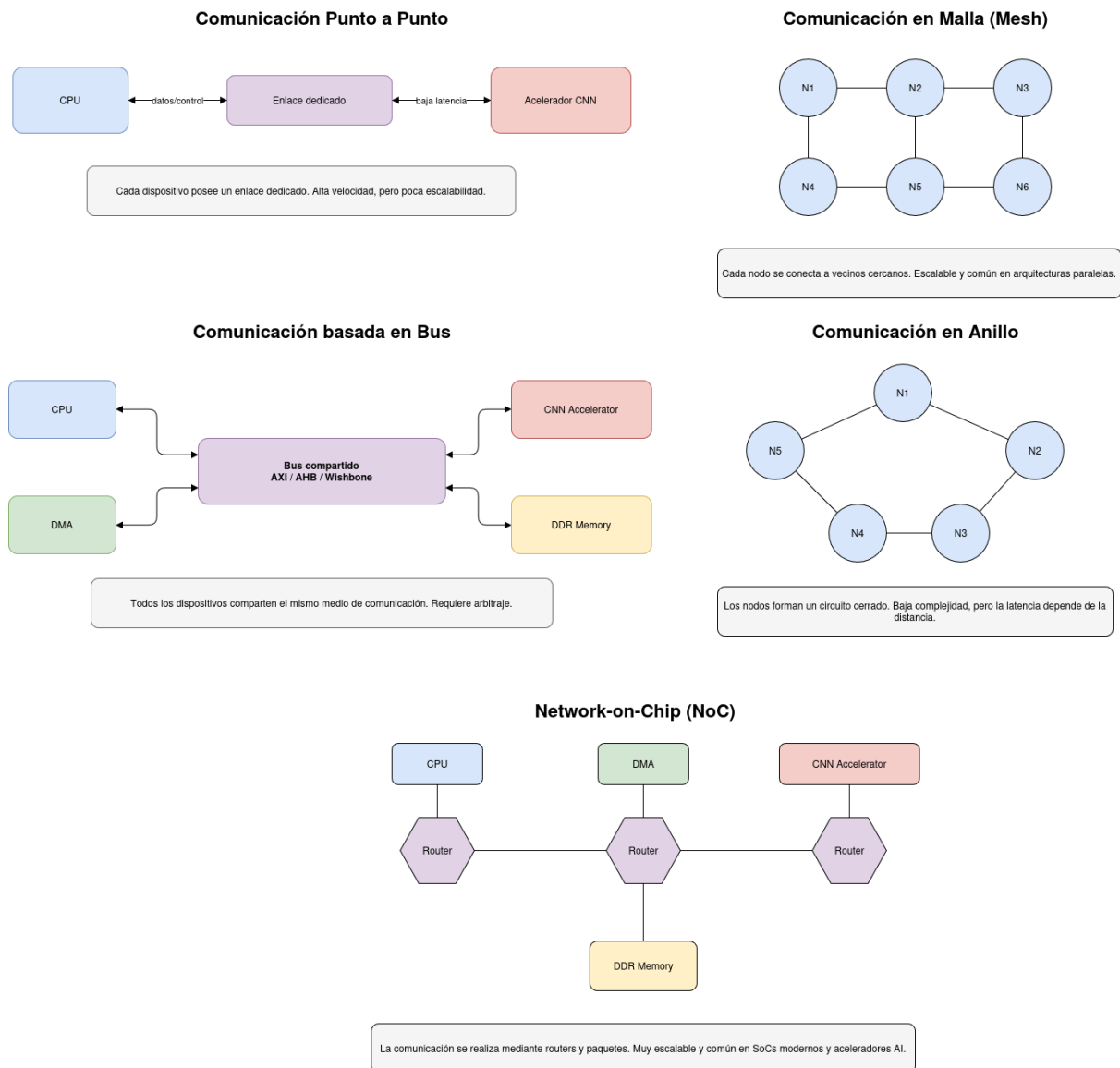


Figura 6. Modelos arquitectónicos para comunicación. Elaborado con ayuda de la IA.

Comunicación por señales

Es el nivel más cercano al RTL y representan las señales dentro de una interfaz, ilustrando la composición e implementación de la interfaz de comunicación.

Ejemplo: clk, reset, valid, ready, addr, data write_enable

Se usa cuando interesa observar ciclos, handshaking y temporización precisa.

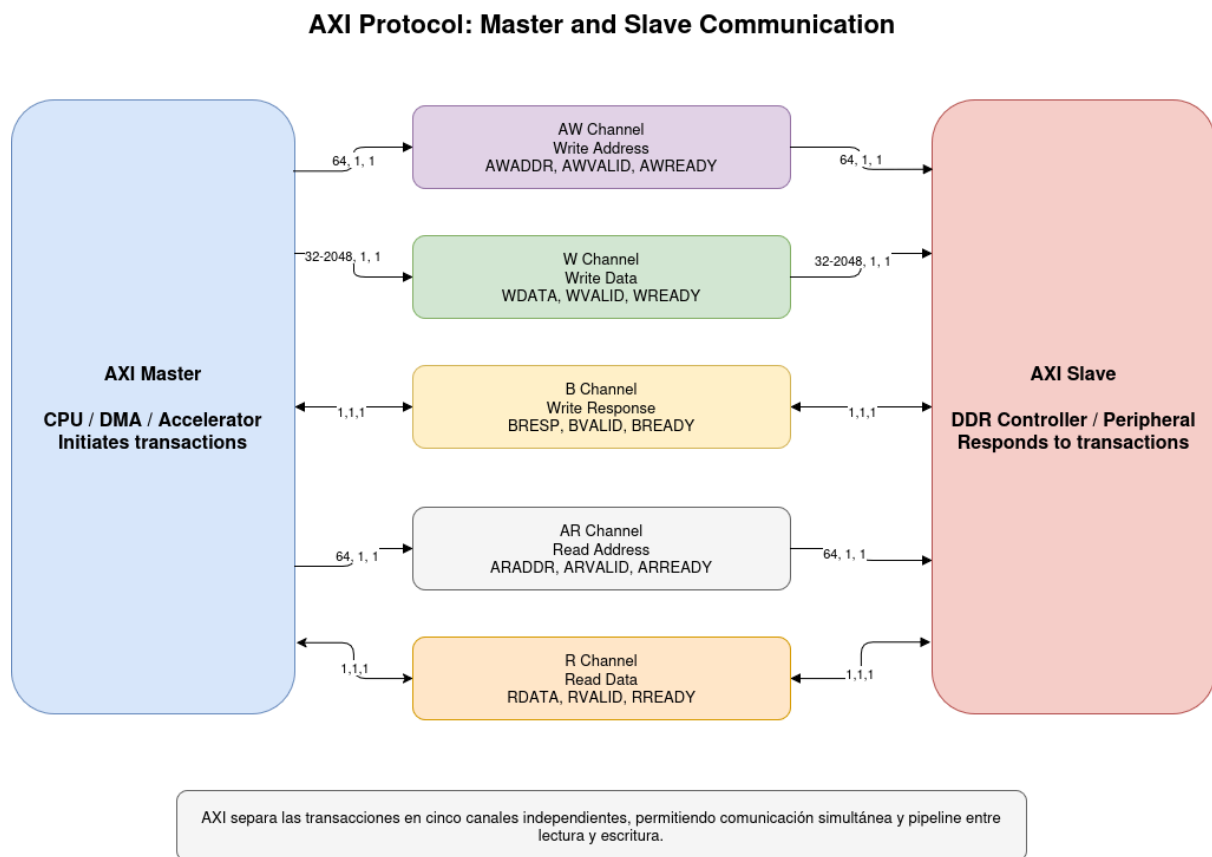


Figura 7. Comunicación AXI. Elaborado con ayuda de la IA.

Métricas arquitectónicas

Los distintos modelos necesitan ser evaluados. Los modelos funcionales nos permiten verificar la funcionalidad algorítmica (que la función sea correcta). Los modelos arquitectónicos nos permiten evaluar que la organización sea correcta + la funcionalidad. Los modelos de ciclo preciso nos permiten evaluar rendimiento + correctitud + funcionalidad. Los RTL nos permite evaluar aspectos de recursos, potencia + rendimiento + correctitud + funcionalidad.

Los modelos permiten evaluar:

Métrica	Descripción	Unidades
Latencia	Tiempo de respuesta del circuito desde que la primer entrada es inyectada hasta la última respuesta.	Ciclos de reloj / tiempo
Throughput	Define cuántas operaciones se realizan en un segundo.	Operaciones por segundo
Área	Recursos hardware	m ² o #celdas
Consumo energético	Energía/potencia	W/J
Escalabilidad	Capacidad de crecer	Número máximo de unidades que se pueden comunicar
Utilización	Uso efectivo de recursos	%

Exploración arquitectónica

El Design Space Exploration (DSE) es el proceso de analizar y comparar múltiples alternativas arquitectónicas para encontrar una solución adecuada según ciertas métricas de diseño. En sistemas electrónicos complejos, existen muchas decisiones posibles:

- Cantidad de núcleos: cuánto puede escalar el sistema de acuerdo con la ley de Amdahl
- Tamaño de memoria: qué tanta latencia se agrega o se involucra en el paginado.
- Ancho de bus: qué tanto se mitiga el memory boundness conforme aumenta el ancho de bus. O si hay problemas de coherencia o de integridad.
- Frecuencia: ¿Cuál es la frecuencia máxima?
- Nivel de paralelismo: qué tanto escala el sistema conforme aumenta el paralelismo, de acuerdo con la ley de Amdahl.
- Aceleradores de hardware: beneficios obtenidos por el offload de tareas, peso de la comunicación.
- Topología de comunicación: generación de congestión y equilibrio de recursos.
- Consumo energético.

El espacio de todas esas combinaciones se conoce como el design space. El objetivo del DSE es explorar ese espacio de manera sistemática.

En general, el DSE nos permite plantear un problema de optimización multiobjetivo, donde se quiere optimizar:

- Rendimiento
- Consumo energético y de recursos
- Latencia
- Throughput
- Costo

Design Space Exploration (DSE) and Optimization Flow

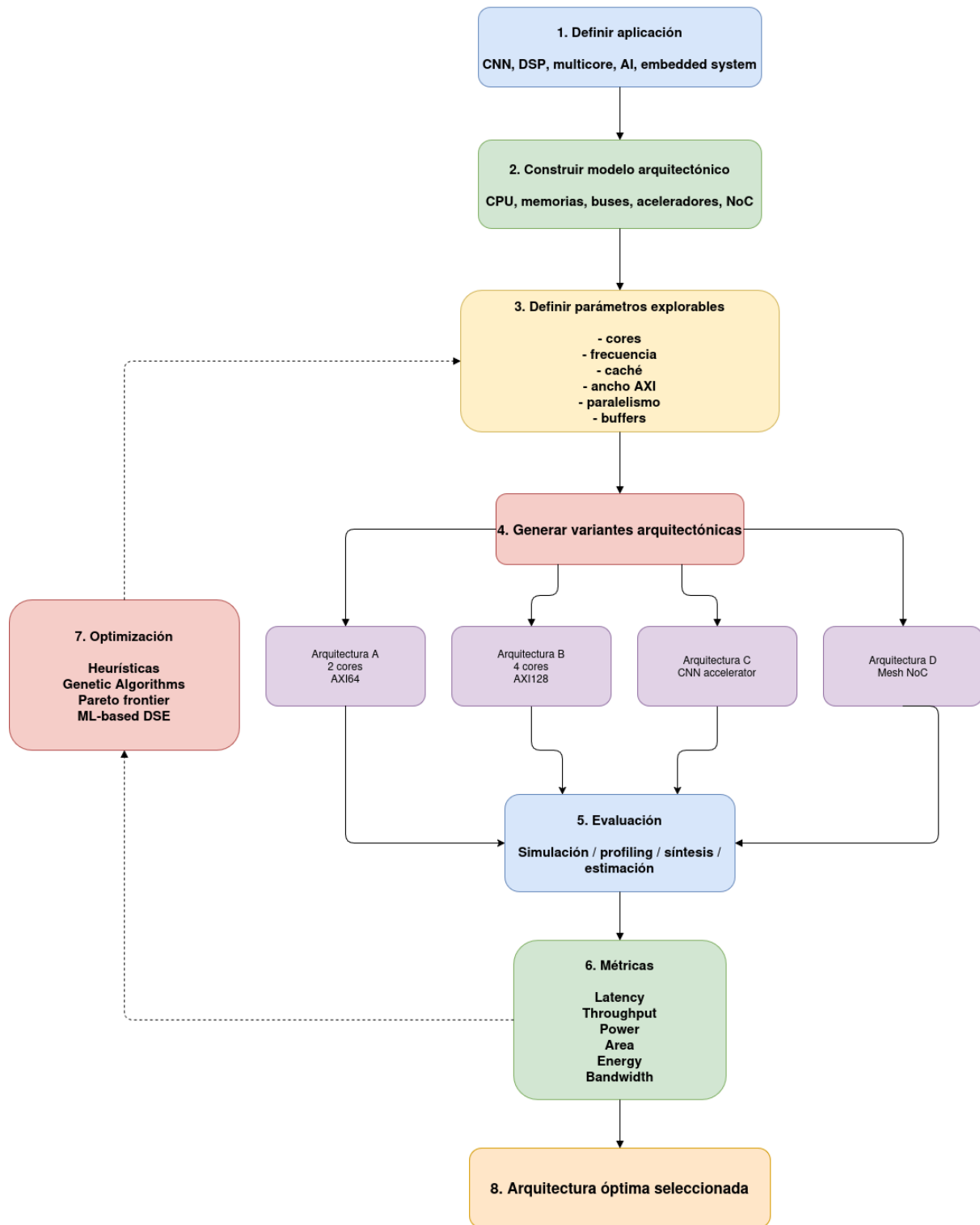


Figura 8. DSE optimisation. Elaborado con ayuda de la IA.

Ahora, existen distintas formas para realizar este proceso de generación de nuevas arquitecturas:

- **Exhaustive Search:** Prueba todas las combinaciones posibles.
- **Heurísticas:** Exploran únicamente regiones prometedoras del espacio de diseño.
- **Algoritmos Genéticos:** La arquitectura se modela como un “individuo”.
- **Machine Learning-Based DSE:** Modelos ML predicen métricas sin sintetizar completamente.
- **Multi-Objective Optimisation:** En sistemas reales no existe una sola métrica.

El objetivo de la exploración es principalmente conseguir un balance o que se favorezca algún recurso particular. También, se genera el frente de Pareto:

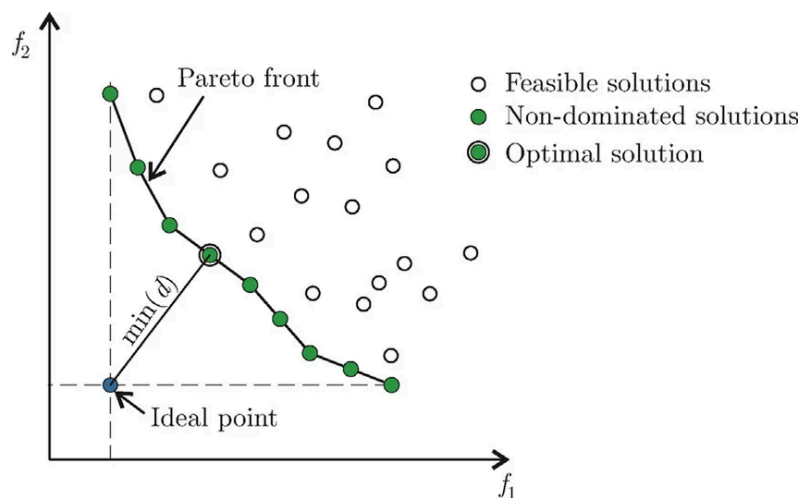


Figura 9. Frente de Pareto. Obtenido de Facundo Bre. 2017. *A computational multi-objective optimization method to improve energy efficiency and thermal comfort in dwellings*

Modelado de IP

Los modelos de *Intellectual Property* (IP) pueden crearse de dos maneras:

1. Usando modelos descritos desde cero en C/C++/SystemC o en Python.
2. Usando modelos generados automáticamente, como plantillas de procesadores.

Cuando los modelos están escritos en C/C++, pueden hacerse un *wrap* con SystemC para poderlos integrar. Lo mismo sucedería con Python, usando LiteX.

El objetivo del modelado de IP es adaptar un modelo de un tercero o descrito *from scratch* para que cumpla con un modelo de interfaces, como las propuestas por TLM 2.0 y de tiempos (approximate, event based, etc...)

Lo mismo aplica para RTL, donde se pueden tener IP configurables y autogenerados.

Modelos Formales

Los modelos formales son representaciones matemáticas o semánticamente precisas de un sistema, utilizadas para describir su comportamiento de manera rigurosa y verificable.

A diferencia de modelos informales o diagramas conceptuales, un modelo formal tiene reglas bien definidas que permiten analizar comportamiento, demostrar propiedades, detectar errores, verificar correctitud y automatizar validación.

Un lenguaje apto para modelos formales y verificación deben permitir:

- Chequeo de propiedades
- Interface checking
- Aserciones

Esto será tema del próximo curso.

Tareas

1. Preparación de exposición sobre patrones de diseño top-down y bottom-up
2. Lectura del proyecto (a ser entregado). Investigación preliminar del tema y nivel 1 de diseño (levantamiento de requerimientos del sistema, entradas y salidas).

Referencias

Brian Bailey, Grant Martin. 2010. ESL Models and their Application